

DATA WRANGLING

In [41]:

```
data = pd.DataFrame(np.arange(6).reshape((2, 3)),
index=pd.Index(['Ohio', 'Colorado'], name='state'),
columns=pd.Index(['one', 'two', 'three'],
name='number'))
data
```

Out[41]:

	number	one	two	three
state				
Ohio	0	0	1	2
Colorado	3	3	4	5

In [43]:

```
result = data.stack()
print(result)
result.unstack()
```

```
state    number
Ohio     one      0
         two      1
         three     2
Colorado one      3
         two      4
         three     5
dtype: int64
```

Out[43]:

	number	one	two	three
state				
Ohio	0	0	1	2
Colorado	3	3	4	5

In [45]:

```
df = pd.DataFrame({'key': ['foo', 'bar', 'baz'],
'A': [1, 2, 3],
'B': [4, 5, 6],
'C': [7, 8, 9]})
df
```

Out[45]:

	key	A	B	C
0	foo	1	4	7
1	bar	2	5	8
2	baz	3	6	9

In [46]:

```
melted = pd.melt(df, ['key'])
melted
```

Out[46]:

	key	variable	value
0	foo	A	1
1	bar	A	2
2	baz	A	3
3	foo	B	4
4	bar	B	5
5	baz	B	6
6	foo	C	7
7	bar	C	8
8	baz	C	9

In []:

```
import pandas as pd

# Assign data
data = {'Name': ['Jai', 'Princi', 'Gaurav',
                'Anuj', 'Ravi', 'Natasha', 'Riya'],
        'Age': [17, 17, 18, 17, 18, 17, 17],
        'Gender': ['M', 'F', 'M', 'M', 'M', 'F', 'F'],
        'Marks': [90, 76, 'NaN', 74, 65, 'NaN', 71]}

df = pd.DataFrame(data)
df
```

Out[]:

	Name	Age	Gender	Marks
0	Jai	17	M	90
1	Princi	17	F	76
2	Gaurav	18	M	NaN
3	Anuj	17	M	74
4	Ravi	18	M	65
5	Natasha	17	F	NaN
6	Riya	17	F	71

In []:

```
#fill the missing values
c = avg = 0
for ele in df['Marks']:
    if str(ele).isnumeric():
        c += 1
        avg += ele
avg /= c

# Replace missing values
df = df.replace(to_replace="NaN",
               value=avg)
df
```

Out[]:

	Name	Age	Gender	Marks
0	Jai	17	M	90.0
1	Princi	17	F	76.0
2	Gaurav	18	M	75.2
3	Anuj	17	M	74.0

4	Name	Age	Gender	Mark
5	Natasha	17	F	75.2
6	Riya	17	F	71.0

In [8]:

```
import pandas as pd
import numpy as np
String_data = pd.Series(['aardvark', 'artichoke', np.nan, 'avocado'])
String_data
```

Out[8]:

```
0    aardvark
1    artichoke
2         NaN
3     avocado
dtype: object
```

In [9]:

```
String_data.isnull()
```

Out[9]:

```
0    False
1    False
2     True
3    False
dtype: bool
```

In [10]:

```
String_data[0] = None
String_data.isnull()
```

Out[10]:

```
0     True
1    False
2     True
3    False
dtype: bool
```

In [11]:

```
from numpy import nan as NA
data = pd.Series([1, NA, 3.5, NA, 7])
data.dropna()
```

Out[11]:

```
0    1.0
2    3.5
4    7.0
dtype: float64
```

In [12]:

```
data = pd.DataFrame([[1., 6.5, 3.], [1., NA, NA],
.....: [NA, NA, NA], [NA, 6.5, 3.]])
cleaned = data.dropna()
data
```

Out[12]:

	0	1	2
0	1.0	6.5	3.0
1	1.0	NaN	NaN

```
2 NaN NaN NaN
0
3 NaN 6.5 3.0
```

In [13]:

```
data[4] = NA
data
```

Out[13]:

	0	1	2	4
0	1.0	6.5	3.0	NaN
1	1.0	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	6.5	3.0	NaN

In [14]:

```
df = pd.DataFrame(np.random.randn(7, 3))
df.iloc[:4, 1] = NA
df.iloc[:2, 2] = NA
df
```

Out[14]:

	0	1	2
0	-0.260732	NaN	NaN
1	0.526847	NaN	NaN
2	0.017321	NaN	0.531534
3	-0.752444	NaN	-0.853585
4	-0.281223	-0.836164	0.706368
5	1.349068	1.732472	-0.075129
6	0.717544	-1.127148	0.072644

In [16]:

```
df.fillna(0)
```

Out[16]:

	0	1	2
0	-0.260732	0.000000	0.000000
1	0.526847	0.000000	0.000000
2	0.017321	0.000000	0.531534
3	-0.752444	0.000000	-0.853585
4	-0.281223	-0.836164	0.706368
5	1.349068	1.732472	-0.075129
6	0.717544	-1.127148	0.072644

DATA TRANSFORMATION

In [23]:

```
data = pd.DataFrame({'k1': ['one', 'two'] * 3 + ['two'],
                      'k2': [1, 1, 2, 3, 3, 4, 4]})
print(data)
data.duplicated()
```

```
      k1  k2
0  one   1
1  two   1
2  one   2
3  two   3
4  one   3
5  two   4
6  two   4
```

Out[23]:

```
0    False
1    False
2    False
3    False
4    False
5    False
6     True
dtype: bool
```

In [24]:

```
data.drop_duplicates()
```

Out[24]:

	k1	k2
0	one	1
1	two	1
2	one	2
3	two	3
4	one	3
5	two	4

In [29]:

```
data = pd.DataFrame({'food': ['bacon', 'pulled pork', 'Bacon',
                              'Pastrami', 'corned beef', 'Bacon',
                              'pastrami', 'honey ham', 'nova lox'],
                     'ounces': [4, 3, 12, 6, 7.5, 8, 3, 5, 6]})
data
```

Out[29]:

	food	ounces
0	bacon	4.0
1	pulled pork	3.0
2	Bacon	12.0
3	Pastrami	6.0
4	corned beef	7.5
5	Bacon	8.0
6	pastrami	3.0
7	honey ham	5.0
8	nova lox	6.0

In [30]:

```
lowercased = data['food'].str.lower()
lowercased
```

Out[30]:

```
0          bacon
1    pulled pork
2          bacon
3      pastrami
4    corned beef
5          bacon
6      pastrami
7    honey ham
8      nova lox
Name: food, dtype: object
```

In [32]:

```
data = pd.Series([1., -999., 2., -999., -1000., 3.])
print(data)
data.replace(-999, np.nan)
```

```
0          1.0
1    -999.0
2          2.0
3    -999.0
4   -1000.0
5          3.0
dtype: float64
```

Out[32]:

```
0          1.0
1         NaN
2          2.0
3         NaN
4   -1000.0
5          3.0
dtype: float64
```

In [33]:

```
ages = [20, 22, 25, 27, 21, 23, 37, 31, 61, 45, 41, 32]
bins = [18, 25, 35, 60, 100]
cats = pd.cut(ages, bins)
cats
```

Out[33]:

```
[(18, 25], (18, 25], (18, 25], (25, 35], (18, 25], ..., (25, 35], (60, 100], (35, 60], (3
5, 60], (25, 35]]
Length: 12
Categories (4, interval[int64, right]): [(18, 25] < (25, 35] < (35, 60] < (60, 100]]
```

In [35]:

```
print(cats.codes)
cats.categories
```

```
[0 0 0 1 0 0 2 1 3 2 2 1]
```

Out[35]:

```
IntervalIndex([(18, 25], (25, 35], (35, 60], (60, 100]], dtype='interval[int64, right]')
```

In [36]:

```
data = pd.DataFrame(np.random.randn(1000, 4))
data.describe()
col = data[2]
col[np.abs(col) > 3]
```

Out[36]:

```
56    -3.264502
163     3.292536
763     3.035598
803    -3.438694
```

Name: 2, dtype: float64

In [37]:

```
df = pd.DataFrame(np.arange(5 * 4).reshape((5, 4)))
sampler = np.random.permutation(5)
sampler
```

Out[37]:

```
array([2, 4, 1, 3, 0])
```

In [39]:

```
print(df)
df.take(sampler)
```

```
      0  1  2  3
0  0  1  2  3
1  4  5  6  7
2  8  9 10 11
3 12 13 14 15
4 16 17 18 19
```

Out[39]:

	0	1	2	3
2	8	9	10	11
4	16	17	18	19
1	4	5	6	7
3	12	13	14	15
0	0	1	2	3

In [40]:

```
df = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'b'],
                    'data1': range(6)})
pd.get_dummies(df['key'])
```

Out[40]:

	a	b	c
0	0	1	0
1	0	1	0
2	1	0	0
3	0	0	1
4	1	0	0
5	0	1	0